

RELATÓRIO 04 — HUGGINGFACE EM GPU ALUGADA

M.I.N.D. — Muñdji Intelligent Neural Developer
Adaptação do router e avaliação sobre modelos retreinados

Campo	Valor
Data	3 de Agosto de 2026
Commit	deec79d
Suite	234 passam · 4 saltados
Mudança de contexto	Sem Ollama. Modelos todos do HuggingFace, alguns retreinados, a correr em GPU alugada (vast.ai e afins).
Decisão sobre os modos	Os quatro suportados, escolhidos por MODEL_MODE, conforme pediste

1. O que fiz

O *model_router.py* assumia Ollama por omissão e foi reescrito. Os quatro modos coexistem atrás do mesmo *ModelRouter* — o resto do sistema continua sem saber qual está activo, que é o ponto de a abstracção existir.

Modo	Quando usar	Notas
openai_compat (novo por omissão)	Servidor vLLM / TGI / llama.cpp na GPU alugada	O servidor gere a GPU; o MIND só fala HTTP e não precisa de torch . Adaptadores LoRA são servidos como nomes de modelo distintos.
hf_local	O MIND corre na própria máquina da GPU	transformers em processo, com importação preguiçosa. Aceita ids do Hub e caminhos de disco. Dependências em requirements-hf.txt.
hf_api	Inference API / Inference Endpoints da HuggingFace	Não exige GPU; em contrapartida os dados saem para a HF.
ollama	Instalações locais existentes	Mantido por compatibilidade.

O campo `*_MODEL` aceita indistintamente **id do Hub** (*Qwen/Qwen2.5-Coder-32B-Instruct*), **nome de adaptador LoRA** (*cortex-lora-v3*) ou **caminho de disco** (*/models/finetuned/cortex*) — é só uma string que o servidor ou o transformers resolvem. Foi assim que os modelos retreinados ficaram suportados sem código especial.

1.1 O que a GPU alugada obrigou a acrescentar

- **Autenticação por componente** (`CORTEX_TOKEN`, `NEURON_N_TOKEN`, ou o global `MODEL_AUTH_TOKEN`). Um servidor de inferência alugado e exposto sem token é uma GPU que estás a pagar para servir outra pessoa — e um registo dos teus prompts nas mãos dela.
- **Componentes podem apontar para servidores diferentes**, o que permite pôr o CORTEX numa instância maior e os NEURONS noutra mais barata.
- **MODEL_ENDPOINT por omissão**, herdado por quem não define o seu — configurar oito endpoints iguais era ruído.

- **O retry passou a ser mais relevante do que era.** Uma instância alugada pode ser reiniciada ou ficar indisponível por momentos, e isso não pode reprovar um ciclo. Há teste para a recuperação e para o caso oposto — um 401 não melhora com insistência e não se repete.

2. Um defeito que os testes apanharam

Ao escrever os testes do router descobri que **as falhas de chamada a modelo nunca chegavam à SYNAPSE DB**. O código registava-as com `cycle_id=0`, que viola a foreign key para a tabela `cycles`; o insert falhava, o `except` engolia o erro, e as falhas ficavam sem rasto nenhum.

Isto estava lá desde a primeira implementação e passou despercebido precisamente porque o silêncio era total — não havia sintoma. É o tipo de defeito que só aparece quando alguém escreve um teste que afirma "depois de falhar, tem de haver uma linha na base de dados".

Corrigido: o `cycle_id` real é propagado do estado até ao router, através do CORTEX, do CEREBELLUM e dos NEURONS. Passa a haver registo por ciclo e por iteração de cada falha de modelo — o que vai ser directamente útil no piloto, onde as falhas de rede contra uma instância alugada são esperadas.

3. A avaliação que pediste: onde alojar os modelos retreinados

A GPU alugada muda a resposta, porque **o disco da instância é efémero**: pode ser reclamado, e cada máquina nova começa vazia.

Opção	A favor	Contra	Veredicto
Disco da instância alugada	Rápido a carregar	Perde-se com a instância; cada máquina nova exige voltar a treinar ou a copiar	Inviável como única cópia
Disco da tua máquina	Durável e sob o teu controlo	Enviar dezenas de GB para cada instância nova, de cada vez	Viável mas doloroso
Repo privado no HF Hub	Durável, versionado, e uma instância nova puxa-o sozinha com um comando	Os pesos passam pela HuggingFace	Recomendado

O argumento decisivo é operacional: com GPU alugada vais destruir e recriar instâncias com frequência, portanto a fonte de verdade tem de estar num sítio de onde uma máquina nova a consiga puxar sem intervenção. Um repo privado faz isso; um disco efémero não.

Sobre "os pesos passam pela HF": os pesos de um modelo afinado sobre os teus ciclos são menos sensíveis do que os dados que os geraram — e esses, a SYNAPSE DB, continuam a nunca sair. Se ainda assim for inaceitável, a alternativa coerente é armazenamento próprio (S3 privado, MinIO na tua rede), não o disco da instância.

3.1 Recomendação técnica: LoRA em vez de checkpoints completos

Com 8 componentes e uma GPU alugada, servir 8 modelos grandes distintos é caro ou impossível. Se os modelos afinados forem **adaptadores LoRA sobre um mesmo modelo base**:

- o vLLM serve vários adaptadores em simultâneo sobre um só modelo carregado, com custo de memória quase nulo por adaptador;
- cada adaptador é exposto como um nome de modelo distinto, portanto `CORTEX_MODEL=cortex-lora-v3` funciona **sem qualquer alteração ao MIND** — já está suportado;
- os adaptadores pesam megabytes em vez de gigabytes, o que torna o repo privado barato e rápido de puxar;
- o retreino a partir da SYNAPSE DB (via `python main.py export`) produz adaptadores, não modelos inteiros.

É a combinação que melhor encaixa no que já está construído: o `export_to_jsonl` existe precisamente para alimentar este ciclo.

4. O que não fiz — e a cedência que tens de conhecer

A inferência deixa de ser local. Uma instância alugada é uma máquina que controlas, mas que está no datacenter de outra pessoa. Consequência inevitável: os prompts e o código gerado saem do teu hardware. O princípio "tudo local" da SYNAPSE DB mantém-se — a base de dados nunca sai — mas convém que isto fique dito, não assumido. Está registado no código e em docs/decisoões/gpu_alugada.md como cedência consciente.

Não implementado	Porquê
Túnel SSH/WireGuard automático	É configuração de infraestrutura, não de aplicação. A recomendação está documentada: o servidor de inferência a escutar só em localhost do lado remoto, e o túnel a fazer a ponte. Pôr o MIND a gerir túneis seria alargar-lhe o âmbito sem necessidade.
Gestão automática de memória de GPU no modo hf_local	Os 8 componentes não cabem numa GPU se forem modelos grandes distintos. Descarregar e recarregar entre componentes é lento e a solução certa é outra: adaptadores LoRA servidos por vLLM. Implementar troca de modelos seria construir a solução errada bem.
Cliente de HF Hub para puxar modelos privados	O transformers e o vLLM já o fazem nativamente com HUGGINGFACE_API_KEY. Reimplementá-lo não acrescentaria nada.

5. O que vou fazer a seguir

O router está pronto para HuggingFace, mas **isso não substitui o piloto**. O sistema continua sem nunca ter corrido com um modelo verdadeiro, e é essa a única coisa que valida tudo o resto.

Assim que houver uma instância a responder, por esta ordem:

#	Passo	O que produz
1	Apontar MODEL_ENDPOINT a um servidor vLLM com um modelo de código do Hub e correr uma tarefa real	Primeira medição verdadeira: quanto demora um ciclo, e se o esqueleto aguenta respostas reais
2	Medir quantas respostas respeitam o esquema JSON	A SYNAPSE DB já regista cada desvio — sai directamente uma percentagem por modelo, sem trabalho extra
3	Aferir NEURON_TIMEOUT_SECONDS e MUNDJI_SANDBOX_TIMEOUT	Os valores actuais (60s e 30s) são palpites; com modelos reais passam a ser medidos
4	Correr 5-10 tarefas por componente e preencher docs/model_selection_criteria.md	Fecha o critério de selecção e desbloqueia o preenchimento dos campos _MODEL
5	Acumular ciclos até ML_MIN_TRAINING_SAMPLES e treinar o HIPPOCAMPUS; depois afinar por LoRA	Fecha o ciclo de aprendizagem que já está construído mas nunca foi alimentado com dados reais

Do que preciso para arrancar: o URL do servidor de inferência, o token, e o id do modelo. Nada mais — o resto está configurável por .env.

Relatório gerado no fim da etapa. Os modos de serviço estão testados contra respostas simuladas de cada API; nenhum foi ainda exercitado contra um servidor real.